Assignment 4: Relational Database vs. NoSQL Database

DBMS Assignment Series | Project topic: **Online Food Delivery System (QuickBite)**

Name: **Jagan S** | Roll No.: **2501121240031**

1. Objective

Compare relational (SQL) databases with NoSQL databases in terms of scalability, consistency, performance and use cases, explain why organisations choose different database technologies, and support the comparison with real-world examples.

2. Introduction

A **relational database (RDBMS)** stores data in tables made of rows and columns, connected by primary and foreign keys and queried with SQL. A **NoSQL database** ("not only SQL") stores data in other models - documents, key-value pairs, wide columns or graphs - and usually relaxes the rigid schema and some consistency guarantees in exchange for flexibility and horizontal scale. Neither is better in every case; each is a set of design trade-offs.

3. Comparative analysis

Aspect	Relational (SQL)	NoSQL
Data model	Tables with rows and columns; fixed schema; relations via keys	Document, key-value, wide-column or graph; flexible / schema-less
Scalability	Mainly vertical (bigger server). Horizontal scaling (sharding, replicas) is possible but complex, especially for joins	Designed for horizontal scaling: data is partitioned across many commodity nodes
Consistency	Strong - ACID transactions (Atomicity, Consistency, Isolation, Durability)	Often eventual consistency (BASE); many offer tunable or per-document ACID
Performance	Excellent for complex queries, joins and aggregates; speed drops with huge multi-table joins at massive scale	Very fast simple reads/writes by key; high throughput; complex cross-collection queries are weak
Query language	SQL (standard, powerful)	Database-specific APIs/query languages (e.g. MongoDB query, CQL)
Schema changes	ALTER TABLE; changes need planning and migrations	Add new fields to documents at any time
Relationships	Native - foreign keys and joins enforce integrity	Usually denormalised/embedded; no joins or limited joins
Typical examples	MySQL, PostgreSQL, Oracle, SQL Server	MongoDB, Cassandra, Redis, DynamoDB, Neo4j
Best use cases	Banking, ERP, inventory, order processing, booking systems	Big data, real-time apps, caching, IoT, social feeds, product catalogs, recommendation graphs

4. Detailed discussion

4.1 Scalability

Relational systems traditionally scale *up*: a more powerful machine. They can scale out through read replicas, partitioning and sharding, but joins and transactions across shards are hard. NoSQL systems were built to scale *out*: nodes are added and data is spread automatically, which suits workloads that grow to millions of users or terabytes of data.

4.2 Consistency

RDBMSs follow **ACID**: a bank transfer either completely happens or not at all, and all users see correct data. Many NoSQL systems follow **BASE** (Basically Available, Soft state, Eventually consistent). By the **CAP theorem**, a distributed system facing a network partition must choose between consistency and availability; many NoSQL stores choose availability so that the service stays up and replicas converge shortly afterwards.

4.3 Performance

Relational engines are highly optimised for complex analytical queries, multi-table joins, indexing and aggregation. NoSQL stores deliver very low latency and high throughput for simple access patterns (read/write by key or by document), because data that is used together is stored together and no joins are needed. Performance therefore depends on the workload, not on the technology alone.

4.4 Use cases

Choose a relational database when data is structured, relationships matter and correctness is critical (finance, inventory, orders, hospital and student records). Choose NoSQL when data is large, changing in shape or semi-structured (user activity logs, sensor streams, content feeds, caching, social graphs).

5. Why organisations choose different database technologies

- **Nature of the data:** structured and related data fits tables; varied or nested data fits documents or graphs.
- **Consistency requirements:** money, stock and medical data need strong consistency; likes, views and feeds can tolerate brief delays.
- **Scale and growth:** a neighbourhood shop needs one reliable server; a global streaming service needs thousands of nodes.
- **Speed of development and change:** flexible schemas help rapidly evolving products and start-ups.
- **Query needs:** reporting and ad-hoc analysis favour SQL; fixed high-volume access patterns favour NoSQL.
- **Skills, cost and ecosystem:** SQL is widely known, mature and supported by tools; a team may already run a specific platform.
- **Polyglot persistence:** large organisations often use several databases together, each for the job it does best.

6. Real-world examples

Organisation / system	Technology type	Reason for the choice
Banks, payment systems, ERP (e.g. systems on Oracle, PostgreSQL, SQL Server)	Relational	Money transfers need ACID transactions and strict integrity.
Airbnb, Shopify-style e-commerce back-ends (MySQL)	Relational	Orders, customers and payments are highly related, structured data.
Netflix (Apache Cassandra for large-scale data)	NoSQL (wide-column)	Massive write volume across regions with high availability.
Amazon (DynamoDB for shopping-cart style workloads)	NoSQL (key-value)	Single-digit millisecond lookups at huge scale.
Social networks, recommendations (graph databases such as Neo4j)	NoSQL (graph)	Queries follow relationships - 'friends of friends' - faster than many joins.
Web caching and session stores (Redis)	NoSQL (key-value, in-memory)	Very fast temporary data access.

Note: Company examples are illustrative of publicly known usage; companies commonly use more than one database technology.

7. Relevance to the Online Food Delivery System project

The core of QuickBite - customers, orders, order items, payments and delivery fees - is **structured, highly related and needs correct transactions**: an order and all its lines must be saved together or not at all, and a customer must never be billed twice. A relational database is therefore the right choice, and the project uses one with primary/foreign keys and constraints. NoSQL could complement it for other parts of a real delivery platform: restaurant menus with varying options as documents (MongoDB), the live GPS location of delivery partners updated every few seconds (Redis or Cassandra), customer reviews and search logs, and recommendation graphs. A flexible menu record might look like this in a document database:

```
{
  "restaurant": "Pizza Palace",
  "area": "Koramangala",
  "items": [
    { "name": "Margherita Pizza", "price": 299, "veg": true,
      "options": { "size": ["Regular", "Large"], "extra_cheese": true } },
    { "name": "Cold Coffee", "price": 99, "veg": true }
  ]
```

```
}
```

In the relational design the same information is spread across *restaurants* and *menu_items* and joined when needed, which prevents duplication and keeps prices and orders consistent.

8. Conclusion

Relational databases offer strong consistency, a powerful standard query language and enforced relationships; NoSQL databases offer flexible schemas and easy horizontal scaling. The right choice depends on the requirements - data structure, consistency, scale and query patterns - rather than on fashion. For this project's transactional order data, an RDBMS is the better fit, with NoSQL as a possible supplement for tracking, caching and flexible content.